

Embedded System Lab Manual

LAB EXPERIMENT NO. 3

Buttons as Inputs STM32 Programming with STM32F103C8T6 Blue Pill C++

Submitted to :

Mr Yasir

Submitted by:

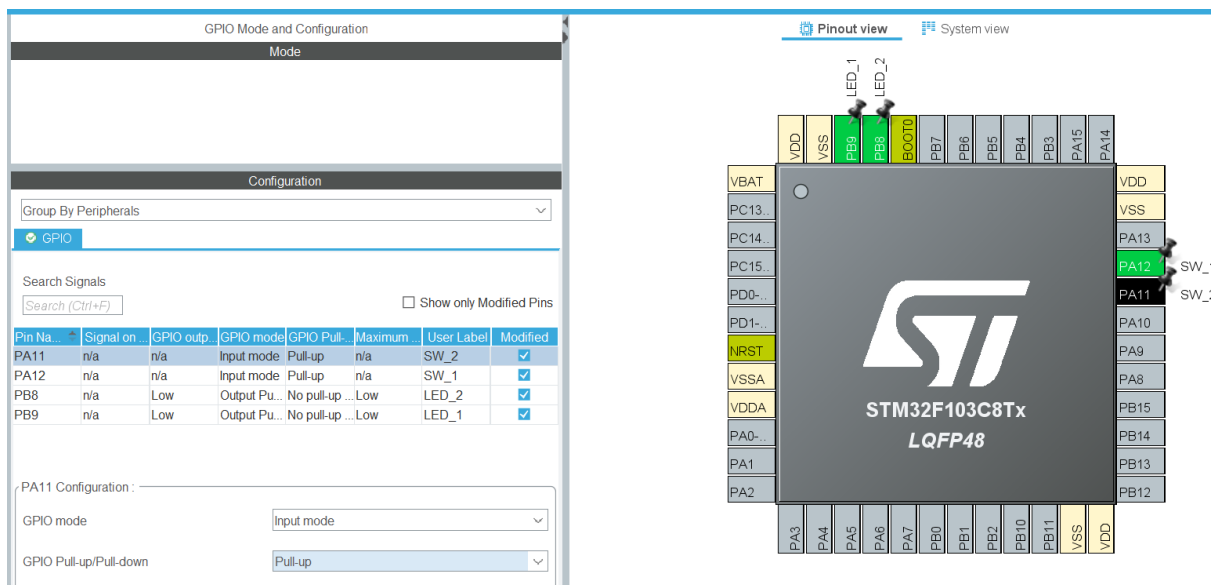
Ahsan Basharat

Date of Submission:

5/07/2027

Introduction

In this lab, we used a push button as an input to control the blinking of LEDs. First, we created a new project named Lab_3 in STM32CubeMX and selected the STM32F103C8T6 microcontroller. Next, we configured two GPIO pins as GPIO Input and enabled the Pull-Up mode for both pins. A Pull-Up configuration keeps the input pin at a HIGH (logic 1) state by default. When the push button is pressed (connected to ground), the pin changes to a LOW (logic 0) state, allowing the microcontroller to detect the button press. After configuring the GPIO pins, we set the system clock source to HSI (High-Speed Internal Oscillator) and configured it to operate at the maximum supported frequency. Finally, we generated the project code using STM32CubeMX and implemented the program to blink the LEDs based on the button input.



CODE and Implementation

First we create a class called Din means digital input and this class public with our GPIO class and then make function called Read. The read() function is used to read the current I/P state of an input GPIO pin. It checks whether the pin is HIGH (1) or LOW (0).

```
class Din : public GPIO{
public:
    Din(GPIO_TypeDef
    *GPIOx,
        uint16_t
        GPIO_Pin);

    ~Din();
    GPIO_PinState
    read(void);

private:
    GPIO_TypeDef * GPIOx;
    uint16_t GPIO_Pin;

};
```

Call class on Source file

We call Constructor with parameters and then call destructor and then make a read function

GPIO_PinState is the return type of the function.

It means the function returns a value of type GPIO_PinState.

```
GPIO_PinState Dout::read(void)
{
    return HAL_GPIO_ReadPin(_GPIOx, _GPIO_Pin);
}
Din::Din(GPIO_TypeDef *GPIOx,
        uint16_t GPIO_Pin)
    : _GPIOx(GPIOx), _GPIO_Pin(GPIO_Pin)
{
}
Din::~Din(){}
}
GPIO_PinState Din::read(void){
    return HAL_GPIO_ReadPin(_GPIOx, _GPIO_Pin);
}
```

Main.cpp

In the main function we call the functions and write condition.

First we initialize the functions with parameters

- Dout led1(LED_1_GPIO_Port, LED_1_Pin);
- Dout led2(LED_2_GPIO_Port, LED_2_Pin);
- Din SW_1(SW_1_GPIO_Port, SW_1_Pin);
- Din SW_2(SW_2_GPIO_Port, SW_2_Pin);

led1.write(SW_1.read());

This line works in three steps:

- 1) SW_1.read();

Reads the state of Switch 1.

It returns:

- GPIO_PIN_SET → HIGH (1)
- GPIO_PIN_RESET → LOW (0)

```
int main(void)
{
    HAL_Init();

    SystemClock_Config();

    Dout led1(LED_1_GPIO_Port, LED_1_Pin);
    Dout led2(LED_2_GPIO_Port, LED_2_Pin);

    Din SW_1(SW_1_GPIO_Port, SW_1_Pin);
    Din SW_2(SW_2_GPIO_Port, SW_2_Pin);

    while (1)
    {
        led1.write(SW_1.read());
        led2.write(SW_2.read());
    }
    /* USER CODE END 3 */
}
```

2) The returned value is passed to write():

- `led1.write(GPIO_PIN_SET);`

3) `write()` sends this value to the LED pin:

- `HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, state);`
- So the LED pin becomes HIGH or LOW

Simulation

We see that PA12 SW_1 is close and LED at PB9 is on and continuous if we open both then both leds are closed and if we close both switch then all leds are glow

